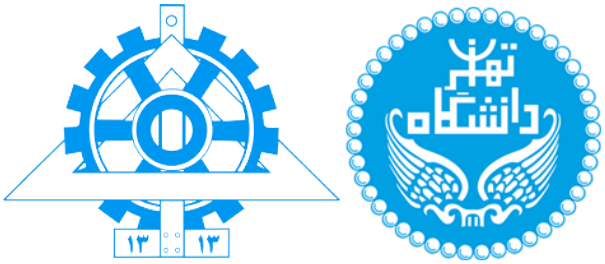

University of Tehran

College of Engineering

School of Electrical and Computer Engineering

VLSI Design

Fourth Computer Assignment

Logic Gate Sizing, Flip-Flops, and Pipelining

Prepared by:

Faezeh Misaghi

Instructor:
Dr. Vahdat

2025-2026

Contents

1 Part One: Computational Circuit Design

1.1 Logic Gate Design and Transistor Sizing

In this section of the project, the goal is to design the logic gates of the combinational circuit and determine their transistor dimensions (sizing). The current driving capability of the pull-up and pull-down branches in all gates must be equivalent to a base inverter, and they are subsequently scaled according to specified scaling factors. The required logic gates are implemented in the `gates.sp` file.

1.1.1 Calculating the Base Inverter Dimensions

For the 180 nm technology, the minimum channel length and width for transistors are considered to be $L_{min} = 180$ nm and $W_{min} = 220$ nm. To equalize the current driving capability of the NMOS and PMOS transistors, the difference in mobility between electrons and holes must be compensated.

By referring to the `mm018.l` library file, the mobility parameter (U_0) values were extracted:

- Electron mobility in NMOS: $\mu_n \approx 0.03727$
- Hole mobility in PMOS: $\mu_p \approx 0.01060$

By calculating the ratio of these two coefficients, β is obtained:

$$\beta = \frac{\mu_n}{\mu_p} = \frac{0.03727}{0.01060} \approx 3.5$$

Based on this, the dimensions of the base inverter transistors were determined as follows:

- $L = 180$ nm (for all transistors in the circuit)
- $W_{n\_base} = 220$ nm
- $W_{p\_base} = \beta \times W_{n\_base} = 3.5 \times 220 = 770$ nm

1.1.2 NAND2 Gate (Scaling Factor: 2.2)

In a 2-input NAND gate, the transistors in the pull-down network are in series. Therefore, to maintain the equivalent resistance, their channel widths must be doubled. The pull-up network transistors are in parallel and retain their base sizes. Finally, the overall dimensions are multiplied by a factor of 2.2.

- NMOS (Series): $2.2 \times (2 \times 220 \text{ nm}) = 968$ nm
- PMOS (Parallel): $2.2 \times (1 \times 770 \text{ nm}) = 1694$ nm

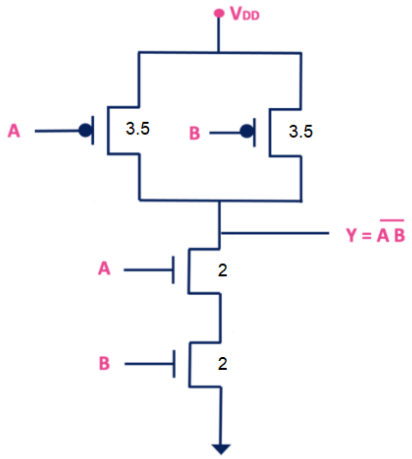

Figure 1: Schematic and sizing of the NAND gate

1.1.3 AND2 Gate (Scaling Factor: 2.4)

This gate is obtained by cascading a NAND2 gate and a NOT gate. The dimensions of both stages (the NAND section and the NOT section) are scaled by this factor based on the structural rules relevant to each.

- NAND2 Section: The NMOS transistors are in series ($2.4 \times 2 \times 220 = 1056$ nm) and the PMOS transistors are in parallel ($2.4 \times 770 = 1848$ nm).
- NOT Section: Scaling is applied directly ($W_n = 528$ nm and $W_p = 1848$ nm).

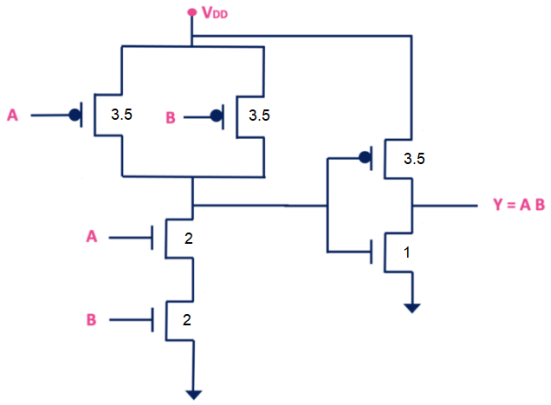

Figure 2: Schematic and sizing of the AND gate

1.1.4 NOR2 Gate (Scaling Factor: 2.8)

In a 2-input NOR gate, the pull-up network transistors are in series, which increases the path resistance; thus, their channel widths must be doubled. The NMOS transistors are in parallel and have the base size. Finally, a factor of 2.8 is applied.

- NMOS (Parallel): $2.8 \times (1 \times 220 \text{ nm}) = 616 \text{ nm}$
- PMOS (Series): $2.8 \times (2 \times 770 \text{ nm}) = 4312 \text{ nm}$

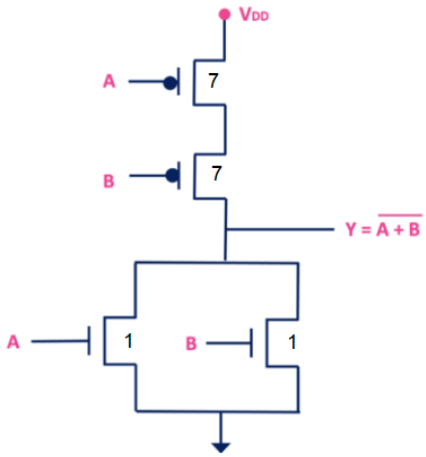

Figure 3: Schematic and sizing of the NOR gate

1.1.5 XOR2 Gate (Scaling Factor: 3.4)

In the static CMOS implementation of the XOR gate, the worst-case path for charging and discharging the output capacitor passes through two series transistors (in both the pull-up and pull-down networks). Therefore, to match the base inverter, the size of both networks is first doubled and then multiplied by a factor of 3.4.

- NMOS (Worst-case path of 2 series transistors): $3.4 \times (2 \times 220 \text{ nm}) = 1496 \text{ nm}$
- PMOS (Worst-case path of 2 series transistors): $3.4 \times (2 \times 770 \text{ nm}) = 5236 \text{ nm}$

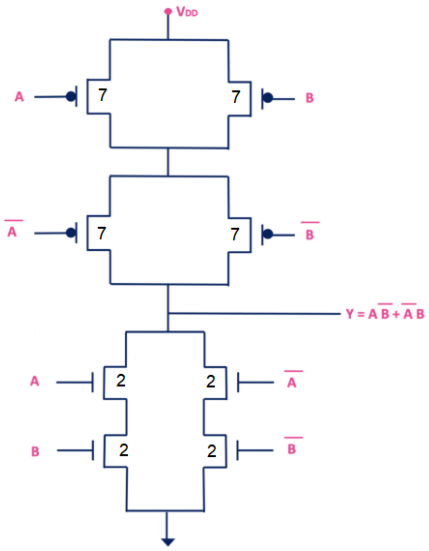

Figure 4: Schematic and sizing of the XOR gate

1.1.6 XNOR2 Gate (Scaling Factor: 3.8)

The structure of the XNOR gate is entirely similar to the XOR gate regarding critical current paths, and its worst-case scenario involves current passing through two series transistors. The calculations are identical to the previous section; only the scaling factor is increased from 3.4 to 3.8.

- NMOS: $3.8 \times (2 \times 220 \text{ nm}) = 1672 \text{ nm}$
- PMOS: $3.8 \times (2 \times 770 \text{ nm}) = 5852 \text{ nm}$

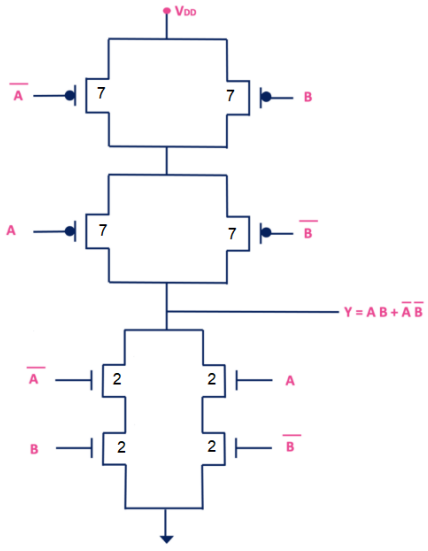

Figure 5: Schematic and sizing of the XNOR gate

1.1.7 NOR3 Gate (Output Gate with Scaling Factor: 5)

This gate has three inputs. The pull-up network consists of three series PMOS transistors. To compensate for the current reduction, the channel width of each transistor must be 3 times that of the base PMOS. The NMOS transistors are in parallel. Finally, the overall size of the gate is scaled to achieve a factor of 5.

- NMOS (Parallel): $5.0 \times (1 \times 220 \text{ nm}) = 1100 \text{ nm}$
- PMOS (Three series transistors): $5.0 \times (3 \times 770 \text{ nm}) = 11550 \text{ nm}$

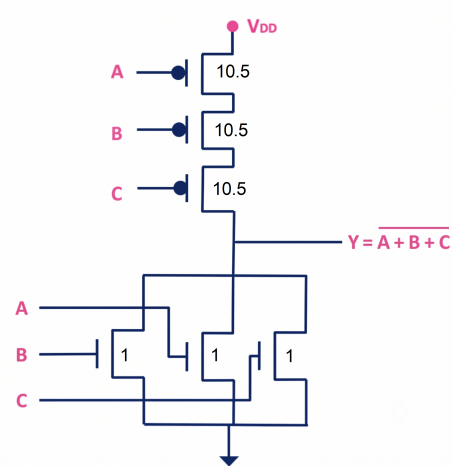

Figure 6: Schematic and sizing of the NOR3 gate

The final dimensions applied to the transistors of each circuit gate are calculated and reported separately in the table below.

Table 1: Final transistor dimensions of the circuit gates

Gate	Scaling Factor	NMOS Size	PMOS Size
NOT	1.2	264 nm	924 nm
NAND2	2.2	968 nm	1694 nm
AND2	2.4	1056 nm, 528 nm	1848 nm
NOR2	2.8	616 nm	4312 nm
XOR2	3.4	1496 nm	5236 nm
XNOR2	3.8	1672 nm	5852 nm
NOR3	5.0	1100 nm	11550 nm

1.2 Investigating and Measuring Worst-Case Circuit Delay

In this section of the project, the primary goal is to calculate the worst-case delay of the circuit and identify the critical path. Due to the structure of logic circuits, applying random inputs can cause the output of some intermediate gates to lock onto a constant value, blocking the signal propagation path. Therefore, to conduct an accurate evaluation and extract the true delay, we utilized the Path Sensitization technique.

To implement this method, we divided the simulation process into four distinct sections and developed four separate testbench files named `A.sp`, `B.sp`, `C.sp`, and `D.sp`. The procedure in each of these files is as follows:

- In each file, only one of the input pins is designated as a pulse signal.
- The other circuit inputs are set to non-controlling values. This means their DC values are chosen such that the logic path from the input under test to the final output remains completely open.

Finally, by independently executing these four files in HSPICE software and using the `.measure` command, the propagation delay of the signal from each input to the output was evaluated. The results of these simulations for both Rise Time and Fall Time are extracted and summarized in the table below:

Input Path	Rising Edge Delay (Rise)	Falling Edge Delay (Fall)
Test A	465.9 ps	543.5 ps
Test B	625.8 ps	560.0 ps
Test C	343.2 ps	453.7 ps
Test D	415.4 ps	315.9 ps

Based on the outputs of the `.measure` command, the worst-case overall circuit delay is 625.8 picoseconds (6.258e-10 seconds). This maximum delay occurs when the constant input values are $A = 1.8\text{V}$, $C = 0\text{V}$, and $D = 0\text{V}$, and a rising edge (transition from 0 to 1) is applied to input B.

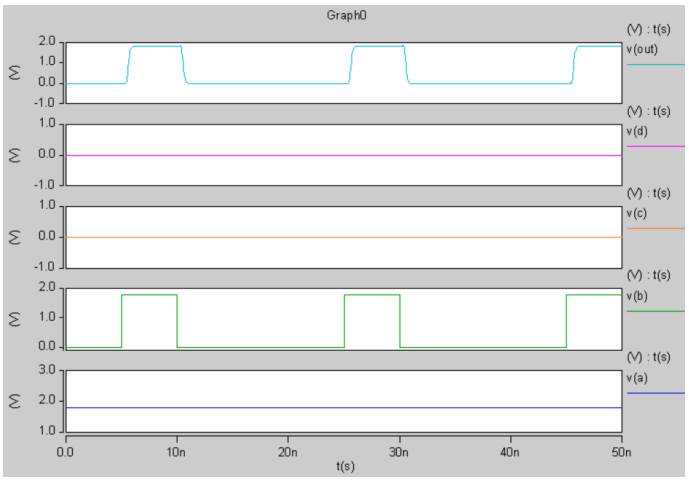

Figure 7: Simulation plot

2 Part Two: Register Design and Placing the Computational Circuit Between Two Registers

2.1 Clock Edge Sensitivity Analysis

The circuit implemented in Figure 3 is a Positive Edge-Triggered Master-Slave flip-flop. The reasoning behind this edge sensitivity is as follows:

- The circuit consists of two sections: Master and Slave. The Master latch uses an inverter in the clock path, making it sensitive to the low level (zero) of the clock, whereas the Slave latch is directly controlled by the main clock and is sensitive to the high level (one).
- When CLK=0, the Master latch is transparent and receives the input data D, but the Slave latch is locked. As soon as the rising edge occurs (transition from 0 to 1), the Master latch locks and holds the instantaneous data, while the Slave latch becomes transparent and transfers the data stored in the Master to the output Q. This combination ensures that the output only updates at the moment the clock transitions, remaining isolated from changes in D at all other times.

2.2 Performance Verification and Sampling Analysis

To verify correct operation, a transient simulation was performed by applying clock and data signals. An example of correct sampling can be observed in the figure below:

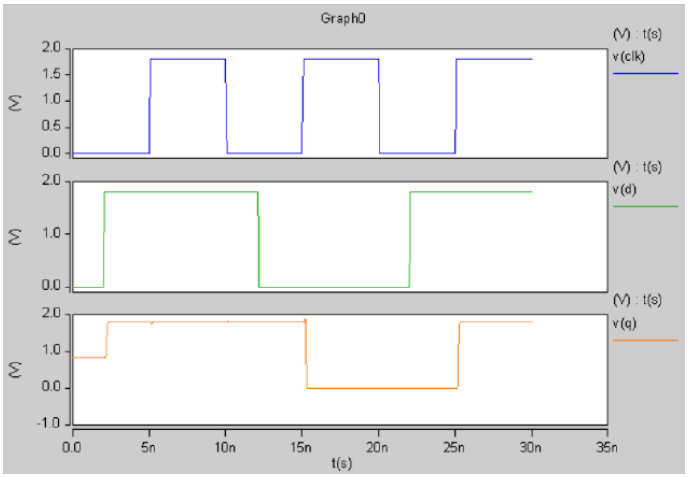

Figure 8: Waveform illustrating the Master-Slave flip-flop operation and how output Q samples input D at the clock's rising edge

As shown in the waveform, at $t = 15$ ns, simultaneously with the clock's rising edge, output Q adopts the value of input data D, which is zero at that moment. Also, at $t = 25$ ns, despite D being high at the edge, output Q transitions to logic high, demonstrating the circuit's sampling precision.

During the intervals when the clock is constant, e.g., 12 ns to 14 ns, even though input D changes from 1 to 0, output Q maintains its previous value. This behavior indicates

the correct operation of the flip-flop in isolating the input from the output until the next clock edge arrives.

2.3 Measuring Master-Slave D-FF Timing Parameters

In this section, the timing characteristics of the designed register were simulated and measured. To make the results more realistic, a 50 fF load capacitor was placed at the register's output. In all simulations, to prevent unstable states at time zero and sudden voltage spikes, the initial conditions of the latch's internal nodes were initialized using the `.IC` command.

2.3.1 Measuring Clock-to-Q Delay (t_{cq})

This parameter represents the time it takes for input changes to appear at the output after the active clock edge arrives. For a precise calculation, signal D was stabilized at a completely safe margin before the clock's rising edge to strictly satisfy the setup time requirement. Then, using the `.MEASURE` command, the time interval between the clock voltage reaching 50% of the supply voltage (0.9V) and the output voltage reaching 50% of the supply voltage was calculated. To report the most accurate result, the worst-case $t_{(cq,LH)}$ and $t_{(cq,HL)}$ were measured separately:

- The value of $t_{(cq,LH)}$ was obtained as 470.2 ps.
- The value of $t_{(cq,HL)}$ was obtained as 497.4 ps.

Based on the principle of considering the worst-case scenario, the largest value is reported as the final flip-flop delay:

$$t_{cq} = 497.4 \text{ ps}$$

2.3.2 Measuring Setup Time (t_{setup})

The setup time is the minimum time the data must remain stable before the active clock edge for correct sampling to occur. To find this critical boundary, the time interval between the data transition edge and the clock edge was defined as a parametric variable (t_{su}). Next, using a swept transient analysis (`.TRAN ... SWEEP`), the data edge was moved closer to the clock edge in 10 ps steps. At larger distances, the output delay was constant. As t_{su} decreased below 50 ps (at 40 ps), the Master latch did not have enough time to stabilize the voltage of its internal nodes, leading to total output failure. The last time the circuit sampled successfully without a significant increase in delay was recorded as the setup time. Final value:

$$t_{setup} = 50 \text{ ps}$$

2.3.3 Measuring Hold Time (t_{hold})

The hold time is the minimum time the data must retain its value after the active clock edge. To measure this, it was assumed the data arrived at the circuit fully satisfying the setup time. Then, by defining a parametric variable (t_h) and sweeping it, the falling edge

of the data was moved from the right closer to the clock edge and even past it (negative values). The simulation showed that the circuit functions correctly up to $t_h = -110$ ps and only fails at -120 ps. The reason for this negative time is the inherent propagation delay of the inverter gates in the clock path inside the flip-flop, which causes the Master latch to remain open for a fraction of a second after the clock becomes zero at the block's input. Final value:

$$t_{hold} = -110 \text{ ps}$$

2.4 Measuring and Calculating the Maximum Operating Frequency

In this section, the goal is to determine the maximum operating frequency (f_{max}) of the circuit when the designed computational circuit is placed between two registers. To achieve this, the frequency was first extracted practically through simulation and then compared with theoretical calculations.

2.4.1 Practical Calculation of Maximum Operating Frequency via Simulation

To obtain the practical operating frequency, a pipeline structure was designed in which inputs enter the first register synchronized with the clock edge, pass through the combinational circuit, and are finally sampled by the output register.

The worst-case delay path of the circuit, test path B, was activated by applying the appropriate stimulus. Then, using a transient analysis and sweeping the clock period, the value of T_{clk} was gradually reduced to identify the boundary of circuit failure.

Simulation results showed that for periods smaller than 1260 ps, the output voltage drops below the standard logic 1 level (90% of the supply voltage, equivalent to 1.62V), causing a sampling error in the circuit. Therefore, the minimum allowable period in practical simulation is as follows:

- Minimum practical period: $T_{min(practical)} = 1260$ ps
- Maximum practical frequency:

$$f_{max(practical)} = \frac{1}{T_{min(practical)}} = \frac{1}{1260 \times 10^{-12}}$$

  The obtained frequency is approximately 793.6 MHz.

2.4.2 Theoretical Calculation of Maximum Operating Frequency

To theoretically calculate the minimum allowable clock period, the maximum delay constraint equation is used:

$$T_{min} = t_{cq} + t_{pd,comb} + t_{setup}$$

By substituting the timing parameters obtained in previous simulations:

- Clock-to-output delay of the flip-flop (t_{cq}): 497.4 ps
- Setup time of the flip-flop (t_{setup}): 50 ps

- Worst-case delay of the combinational circuit ($t_{pd,comb}$): 625.8 ps

$$T_{min(theory)} = 497.4 + 625.8 + 50$$

$$T_{min(theory)} = 1173.2 \text{ ps}$$

Therefore, the theoretical maximum operating frequency of the circuit is:

$$f_{max(theory)} = \frac{1}{1173.2 \times 10^{-12}}$$

The theoretical frequency is approximately 852.3 MHz.

2.4.3 Comparing Theoretical and Practical Frequencies and Investigating the Causes of Discrepancy

Comparing the results reveals that the frequency obtained from practical simulation (793.6 MHz) is lower than the theoretically calculated frequency (852.3 MHz). This difference of roughly 60 MHz is completely logical and occurs at the transistor level due to the following physical reasons:

- In the theoretical calculation, the delays of gates and flip-flops were evaluated in isolation. However, in an integrated practical circuit, the input capacitances of the combinational circuit act directly as a load on the output of the first register, and the input capacitance of the second register also increases the load on the combinational circuit. This connection causes the actual propagation delay to be higher than a simple algebraic sum.

- In independent calculations, signals are assumed to have ideal, sharp edges. In a pipeline circuit, after passing through several logic stages, the signal experiences sluggishness in its rising and falling edges. This issue requires the output register to have a longer setup time to correctly latch the data compared to an isolated state.

- The circuit does not suddenly fail; rather, as it approaches the theoretical frequency boundary, it enters an instability region where the voltage level drops. To maintain signal integrity and bring the output to the standard voltage level, the circuit in practice requires a larger period.

2.5 Designing a Sampling Error Scenario at an Invalid Frequency

In this experiment, the clock period is set to a value much lower than the minimum allowable period. Input signal B (the orange waveform) is applied such that it goes high precisely in time to activate the worst-case delay path of the computational circuit.

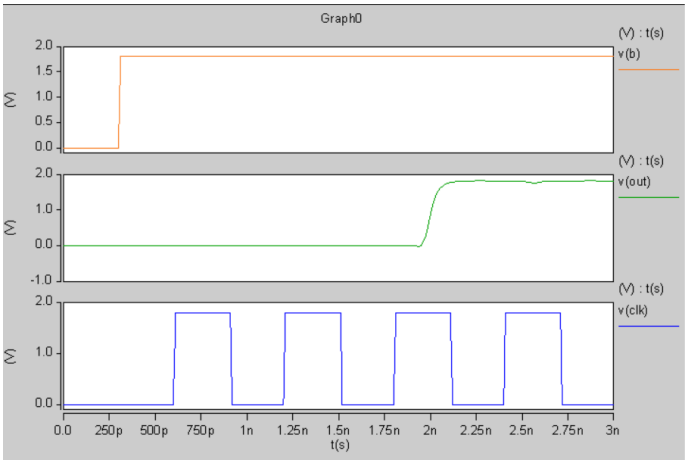

Figure 9: Sampling waveform and simulation of a synchronization error scenario

By examining the output waveforms, the progression of the synchronization disaster in the circuit is clearly observable:

1. First clock edge: The first register samples the input data B and feeds it into the combinational circuit.
2. Propagation delay: Given that the critical path delay of the combinational circuit exceeds the allotted time, the processed data arrives at the input of the output register much later.
3. Second clock edge: The data latching edge in the second register arrives. Since the new data is still on its way and hasn't arrived, the setup time condition is completely violated. At this moment, the output register samples the previous value (zero).
4. Third clock edge: The correct data has finally stabilized. The output register latches the data on this edge, and the final output (green waveform) transitions high.

3 Part Three: Placing the Computational Circuit Between an Array of Latches

3.1 Determining Latch Sensitivity Type (Positive or Negative)

The circuit implemented in Figure 2 is a Negative-Level Sensitive Latch.

In this circuit's structure, the clock signal (CLK) passes through an inverter gate before entering the controlling gates, becoming the CLK_bar signal.

When the clock is at logic low (CLK=0), the CLK_bar signal becomes one. In this state, the input NAND gates are activated, allowing data D and its inverse (D_bar) to change the values of S_bar and R_bar. Consequently, output Q follows the input values of D.

When the clock is at logic high (CLK=1), the CLK_bar signal becomes zero. This forces the outputs of both input NAND gates, S_bar and R_bar, to a logic high state. Applying inputs of 1 and 1 to the cross-coupled SR latch makes the circuit maintain its previous state, and changes in input D have no effect on the output.

Therefore, since the circuit passes data when CLK=0, it is a Negative Latch.

3.2 Necessary Changes to Convert to a Positive-Level Sensitive Latch

For the circuit to exhibit exactly the opposite behavior—meaning it passes data when CLK=1 and locks when CLK=0—the polarity of the clock signal fed to the input NAND gates must be changed.

The required modification at the circuit level is as follows:

- The clock inverter gate must be removed from the input clock path to the X_NAND_S and X_NAND_R gates.
- The CLK signal must be connected directly, without being inverted (NOT), to one of the pins of these two NAND gates.

With this simple change, the input gates are activated only when CLK=1, and the circuit becomes a Positive-Level Sensitive Latch.

3.3 Simulation of Positive and Negative Latches

To verify correct operation, data signal D and clock signal CLK were applied with the same period. According to the obtained timing diagram:

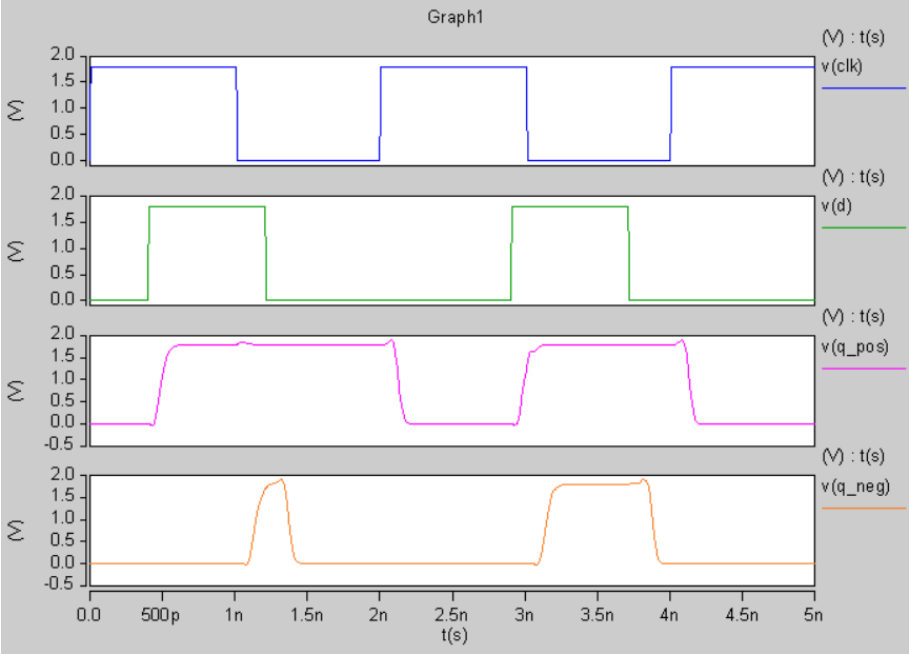

Figure 10: Timing diagram and performance verification of positive and negative latches

- **Positive Latch (q_pos):** This circuit acts transparently during the clock's high phase ($CLK = 1$). It can be observed that in the intervals 0 ns to 1 ns and 2 ns to 3 ns, when the clock is high, output q_pos strictly follows input signal D and transfers its changes to the output. During the clock's low phase ($CLK = 0$), the latch enters a Hold state and maintains the last sampled data value at the output.
- **Negative Latch (q_neg):** This circuit, designed using a clock inverter, exhibits behavior completely complementary to the positive latch. As seen in the diagram, in the intervals 1 ns to 2 ns and 3 ns to 4 ns, when the clock is low ($CLK = 0$), the latch transfers data D to output q_neg. When the clock goes high, the circuit isolates the input and retains its current state.

The simulations clearly demonstrated the logical differences between level-sensitive latches. The positive latch allows data to pass when the clock is high, and the negative latch allows data to pass when the clock is low. The perfect alignment of output changes with the clock control signal levels confirms the correctness of the design and implementation of both structures.

3.4 Simulation, Determining Maximum Operating Frequency, and Analyzing Time Borrowing

3.4.1 1. Maximum Operating Frequency of the Circuit and How It's Measured

To find the maximum operating frequency of the circuit, we incrementally reduced the clock period (T_{clk}) in the HSPICE simulation file until the circuit reached the brink of a timing error and the output latch could no longer sample the correct data stably. Based on the output waveform from this critical simulation, the following values were extracted for the clock signal:

- **Clock Period (T_{clk}):** Considering the clock rising edges at times $t = 0$, $t = 1.5$ ns, and $t = 3.0$ ns, the minimum allowable period for the circuit was found to be 1490 ps.
- **Calculating Maximum Operating Frequency (f_{max}):**

$$f_{max} = \frac{1}{T_{min}} = \frac{1}{1.490 \times 10^{-9}\ \text{s}} \approx 671.14\ \text{MHz}$$

3.4.2 2. Investigating and Analyzing the Occurrence of Time Borrowing

By examining the waveform, it is clear that Time Borrowing has occurred. The timing analysis of the circuit is as follows:

- **Input Change:** Signal v(b) changes at $t = 1.35$ ns. Because the input latch is a Negative Latch, it is fully transparent during the clock's low phase and passes the data immediately.
- **State at Clock Edge ($t = 1.5$ ns):** At the moment of the clock's rising edge, the voltage of the combinational circuit output (v(out_comb)) is still zero volts, meaning the circuit calculations have not finished by the clock edge due to the gates' propagation delay.
- **Time Borrowing Mechanism:** Since the output latch is a Positive Latch, it enters the transparency phase when the clock goes high. As a result, it allows the combinational circuit to continue its calculations even after the clock edge.
- **Amount of Time Borrowed:** The combinational circuit output stabilizes at $t \approx 2.0$ ns, and the output latch passes it through. The circuit has borrowed exactly 500 ps from the active phase of the next clock:

$$\Delta t_{borrow} = 2.0\ \text{ns} - 1.5\ \text{ns} = 500\ \text{ps}$$

3.5 Advantage of Using Three Latches (Breaking the Circuit) Over Two Registers

The most significant advantage of this structure is a **dramatic increase in Maximum Clock Frequency** and, consequently, an increase in system Throughput.

- In the previous case, the signal had to traverse the entire logic path within one clock cycle, meaning the minimum clock period was proportional to the total delay of the entire circuit.
- In the new structure, by applying pipelining techniques, the heavy computational circuit is broken into two smaller sections (CLB_A and CLB_B), with an intermediate latch ($L2$) placed between them. In this scenario, the system's clock period is no longer determined by the total delay, but rather restricted by the slowest critical path (the longer path between CLB_A and CLB_B). Consequently, the minimum period decreases, leading to higher circuit speed.

3.6 CLK1 and CLK2 States Relative to Each Other for Correct Circuit Operation

Since latches are used in this structure, the clock signals must be Non-overlapping Two-Phase Clocks.

- **Performance Analysis:** To prevent Race Conditions, the clocks must be out of phase with each other. In the simplest terms, we must have:

$$CLK2 = \overline{CLK1}$$

- **Importance of Non-overlapping:** If both clocks $CLK1$ and $CLK2$ were active at the same time, input data could pass through the first latch, CLB_A, the second latch, and CLB_B all in one clock cycle, entirely destroying the pipeline logic. Therefore, when $L1$ is sampling and passing data, $L2$ must absolutely be in a locked state, and vice versa. In practical and precise designs, a small dead time is introduced between the edges of these two clocks to ensure they do not overlap.

3.7 Measuring the Worst-Case Delay of the First State

In this section of the project, we divided the main circuit into two separate subcircuits, CLB_A and CLB_B, to investigate the delay of each part independently. For an accurate calculation of the worst-case delay, we used the path sensitization technique; this means that in each test, only one input was treated as a pulse while the rest were set to constant voltages to ensure the signal propagation path to the output remained unblocked.

Worst-Case Delay of Block CLB_A

By simulating all possible paths, it was determined that the worst-case delay for this block is 361.9 picoseconds. This maximum delay occurs when the pulse is applied to input B, while the other inputs (A, C, and D) are held constant at 0 volts. In this state, the critical path extends to the out_xnor output.

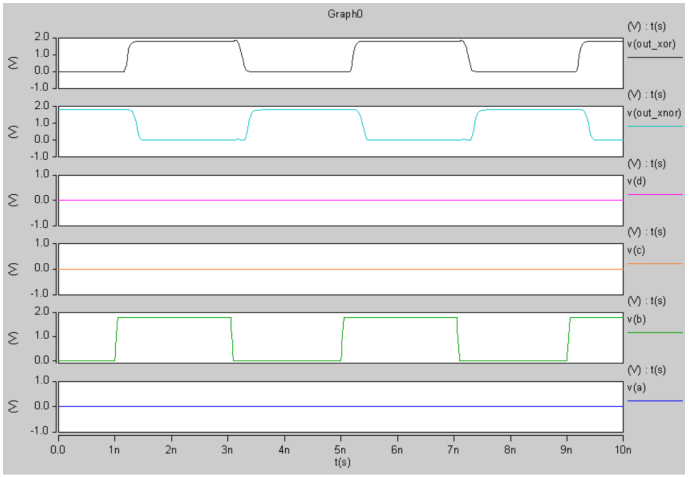

Figure 11: Timing diagram corresponding to the simulation of the worst-case delay for block CLB_A

As seen in the figure above, inputs A, C, and D are locked at zero volts, and input B is oscillating. The out_xnor output inversely follows the changes in input B. Looking closely at the diagram, there is a time difference between the input edge transition and the output reaction; this interval exactly represents the circuit's 361.9 picosecond delay.

Worst-Case Delay of Block CLB_B

For the second block, by evaluating the inputs, the worst-case delay was calculated to be 256.9 picoseconds. This delay occurs when the in_xnor pin is held steady at zero volts, and a variable signal is applied to the in_xor input.

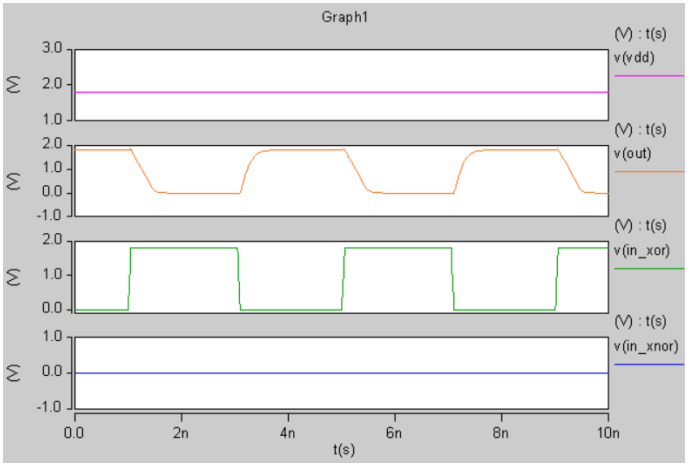

Figure 12: Timing diagram corresponding to the simulation of the worst-case delay for block CLB_B

In this diagram, it can be observed that with in_xnor at zero, the logic path is open for the signal to pass. When a rising edge is applied to the in_xor input, the output voltage drops towards zero. A notable point in this diagram is the sloped edges of the output signal, which are caused by the presence of a 100 fF load capacitor at the output of this block. Charging and discharging this capacitor increases the signal settling time, resulting in the 256.9 picosecond delay.

3.8 Verifying Pipeline Circuit Performance and Time Borrowing

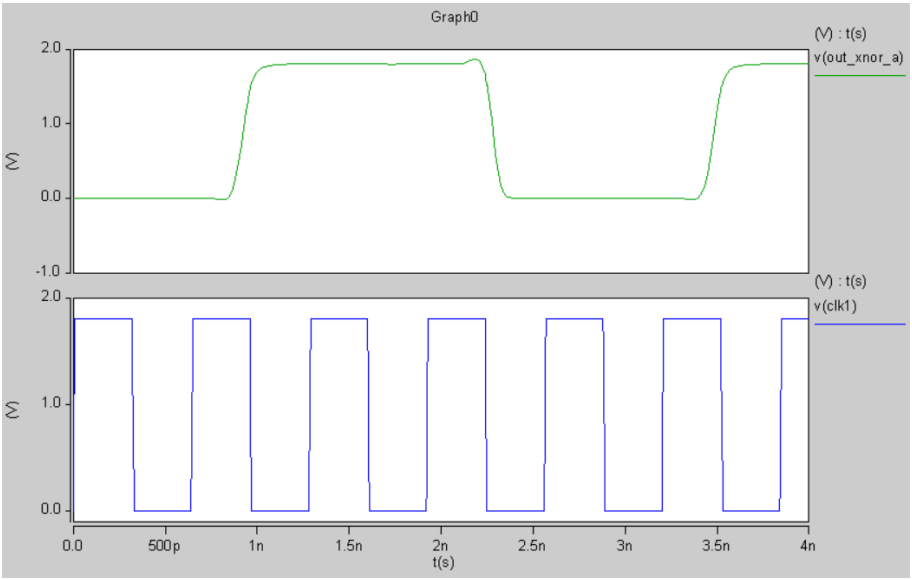

Figure 13: Timing diagram simulating the verification of the pipeline circuit performance and Time Borrowing phenomenon

In this part of the simulation, the circuit's functional correctness was tested at the highest possible operating frequency. Even though we pushed the clock period to its limit, the simulations show that the circuit still produces its final output correctly and the pipeline output successfully tracks input changes. This confirms the functional correctness of the design and the latch connections.

To calculate the amount of time borrowed, we need to know how much the first logic block exceeded its allotted time. The allotted time for the first block to finish its job is exactly at the falling edge of the first clock, CLK1. Therefore, using measurement commands in the software, we calculated the time difference between the clock's falling edge and the moment the first block's output signal, out_xnor_a, reaches its final value. The exact value of this borrowed time, according to the software output, was calculated to be 241.2 picoseconds.

In the diagram above, the blue signal represents the system clock CLK1, and the green signal represents the output of the first block's critical path, out_xnor_a. As is quite clear in the figure, the rising and falling edges of the green signal occur with a significant delay relative to the clock edges. This means that only after the clock phase has ended and the clock has transitioned, the green signal finally reaches its steady-state voltage level. This protrusion and time shift of the signal relative to the clock is exactly the borrowed time. However, because the latch in the next stage is sensitive to the clock voltage level and not its edge, it easily allows this delayed signal to pass through, and the circuit continues to operate without error.

3.9 Finding the Maximum Operating Frequency with Sweep Simulation

To accurately pinpoint the circuit's maximum operating frequency, we selected the critical path (applying a pulse to input B) and used the `SWEEP` command in HSPICE to reduce the clock period (T_{clk}) from 1000 picoseconds down to 300 picoseconds until reaching the circuit's failure point. By reviewing the software's output table, it was determined that the minimum possible period for the correct operation of this pipeline circuit is 410 picoseconds (beyond this point, the circuit experiences timing errors and the final output is not produced). The frequency corresponding to this period, 2.44 GHz, represents the circuit's maximum operating frequency.

Analysis of Operating Frequencies

Comparing the obtained frequencies, it is evident that the pipeline structure designed in this stage achieves an astonishing frequency of 2.44 GHz, which demonstrates a massive increase compared to the previous structures (793.6 MHz and 671.14 MHz).
Reasons for this Frequency Increase:

- **Pipelining Optimization:** The primary reason for this frequency jump is the increased number of pipeline stages. In previous scenarios, the circuit only had two stages, and the critical path delay was entirely contained within a single clock cycle. However, by adding intermediate latches and dividing the logic into three stages, the stage delay has been drastically reduced, which directly boosts the clock frequency.
- **Combined Effect of Time Borrowing and Pipelining:** In this circuit, aside from logic segmentation, the time borrowing feature is also utilized. This technique allows the hard timing constraints that existed in the register-based design to become flexible, pushing the operating frequency to the absolute maximum limit allowable within the manufacturing technology.

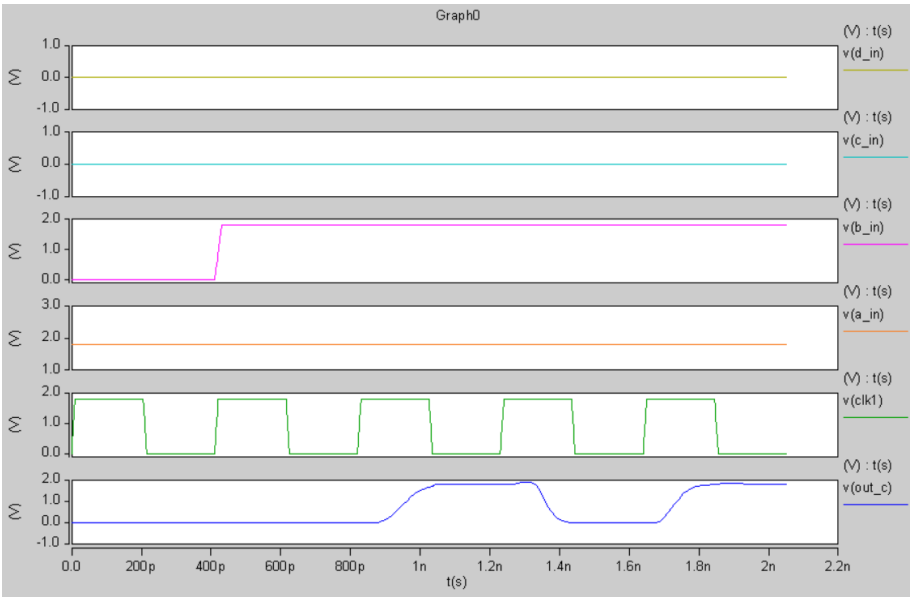

Figure 14: Analysis plot of the pipeline circuit's critical path

3.10 Timing and Performance Analysis of the Pipeline Circuit in Figure 6

All timing simulations and delay extractions for this section were performed similarly to the methodology and structure described in Figure 5. In this analysis, the dynamic behavior of the pipeline circuit was investigated under various clock frequencies to determine the stability limit of the system.

1. Logic Block Delay Analysis

After investigating all critical paths and various switching conditions, the worst-case computational delays (Critical Paths) in each of the logic blocks were extracted as follows:

- **Logic Block CLB_A:** The worst-case computational delay in this block is 68.11 picoseconds. This delay corresponds to stimulating the input via path B and the rising output of the NOR gate in the block's internal structure (`delay_rise_b_nor`).
- **Logic Block CLB_B2:** The worst-case computational delay in this block is much larger, measuring 478.1 picoseconds. This critical path relates to the in_nand signal and the rising edge of the final circuit output (`delay_rise_nand_out`).

2. Time Borrowing Phenomenon and Clock Variations

Since the circuit employs level-sensitive latches (`D_LATCH_POS`) with two complementary clock phases (CLK1 and CLK2), it has the potential for Time Borrowing.

When the clock period is set to $T_{clk} = 800$ ps, the nominal time allocated to each half-cycle is:

$$\frac{T_{clk}}{2} = 400 \text{ ps}$$

Because the delay of block CLB_B2 (478.1 ps) exceeds its legitimate half-cycle (400 ps), this block cannot complete its computations within the allotted time. As a result, the circuit automatically enters a temporary steady state and borrows the necessary time from the next clock phase.

The exact values extracted from the simulation output file (`.mt0`) at this point are as follows:

Simulation Parameter	**Extracted Value**
Clock Period (T_{clk})	800 ps
Half-cycle Time	400 ps
Time Borrowed (`time_borrowed`)	325.6 ps
Total Circuit Delay (`total_delay`)	1.682 ns

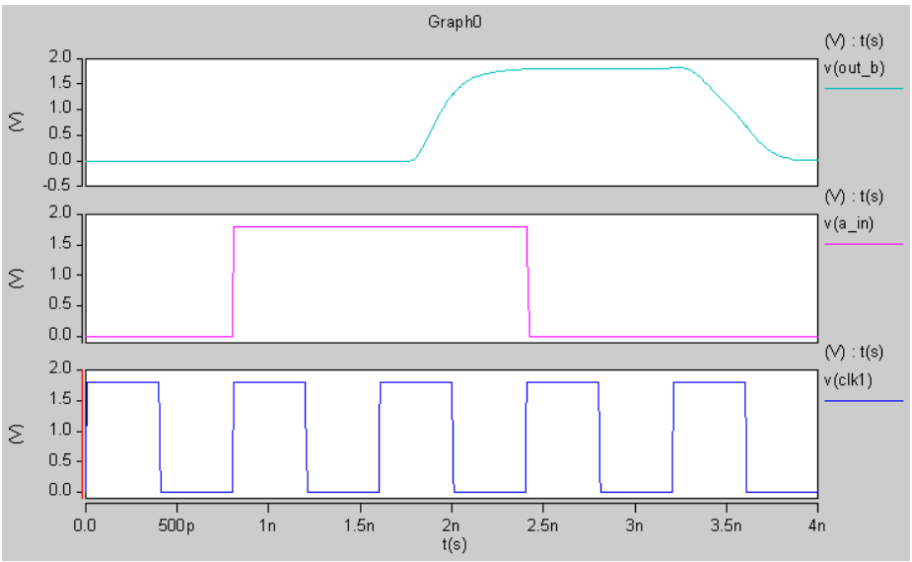

Figure 15: Output waveform of the pipeline circuit simulation at a 800 ps clock

The empirical waveform fully confirms that block CLB_B2 is the time-borrowing segment of the circuit. Although the Time Borrowing mechanism allows latches to compensate for time shortfalls, because this delay is excessively high (`time_borrowed` = 325.6 ps), the data falls behind by one clock cycle, capping the maximum operating frequency of the circuit at this point.

3. Determining Maximum Frequency

By performing a deep sweep over the clock, the operational boundaries of the circuit were determined as follows:

- **True Maximum Frequency ($f_{\mathbf{max}}$):** The last point where the circuit exhibits linear and error-free behavior is at a clock of 820 ps:

$$f_{\max} = \frac{1}{820 \text{ ps}} \approx 1.22 \text{ GHz}$$

As the results demonstrate, the circuit from Figure 5 operates twice as fast as the circuit from Figure 6. The reason for this drastic discrepancy lies in the internal structure of the blocks:

- In the circuit from Figure 5, the computational delay is perfectly balanced and evenly distributed between the stages. Consequently, the circuit easily remains stable at frequencies exceeding 2 GHz.
- In the circuit from Figure 6, the new block CLB_B2 has a very heavy delay (478.1 ps). This block effectively acts as a bottleneck in the critical path, preventing the clock period from going below 820 picoseconds.

This comparison clearly proves that in the design of synchronous digital circuits, the uniform and balanced distribution of delay among pipeline stages (Pipeline Balancing) is of critical importance; ultimately, the overall speed of the system is always dictated by its slowest logic block.